技術検証メモ

作成日	2026年10月6日 13:58
科目名	プログラミング言語

技術検証メモ

概要設計で未確定とした技術選定について、2026年10月時点の公式資料・主要ライブラリを基に事前検証した結果をまとめる。

参照：概要設計

🧪 **実機PoC-A〜Dまで完了。** MobileSAMは条件付き採用、psd-tools・JavaFX/BufferedImage構成・256×256差分タイル履歴は正式採用とし、詳細設計へ移行可能な状態である。

1. 検証結果サマリ

項目	最終判定	採用方針
AIセグメンテーション	採用	MobileSAMを条件付き正式採用。最大3候補の選択UIとMPS失敗時のCPU再試行を実装条件とする
Java↔AI連携	採用	localhost限定のPythonサービスをJavaからHTTPで呼び出す
SAM 3/3.1	今回は対象外	公式実装がCUDA GPU前提・大規模なため不採用
Javaから直接ONNX推論	本線では不採用	公式Java配布物のmacOS ARM64対応が弱いため、7週間では避ける
PSD読み込み	採用	psd-toolsを正式採用。RGBA抽出してRasterLayerへ取り込み、完全なPSD再保存はVersion 1対象外とする
JavaFX画像データ	採用	`BufferedImage.TYPE_INT_ARGB_PRE`を編集側の正本とし、表示用`WritableImage`を再利用する
長時間処理	採用	JavaFX Task / ExecutorServiceでUIスレッドから分離する

項目	最終判定	採用方針
Undo／Redo	採用	Commandパターン＋最大256×256pxの差分タイル。履歴上限は実データ量256MiB

2. AIセグメンテーションモデル

2.1 必要条件

- 画像上をクリックまたは矩形選択して対象物を指定できる
- 対象領域をマスクとして取得できる
- 人物専用ではなく一般物体に対応する
- macOSでローカル実行できる
- 学校課題で利用可能なライセンスである
- 数秒以内を目標に対話的に利用できる

2.2 MobileSAM【条件付き採用】

MobileSAMはSegment Anythingの画像エンコーダを軽量なTiny-ViTへ置き換えたモデルで、点・矩形プロンプトによるセグメンテーションを利用できる。公式リポジトリはONNXエクスポートも提供しており、モデルはApache 2.0ライセンスである。

採用理由

- クリック選択という本システムのUIと機能が一致する
- 元のSAMより大幅に軽量
- CPU実行例があり、GPU必須ではない
- Python実装が単純でPoCしやすい
- ONNX化の逃げ道もある

参考： MobileSAM GitHub

2.3 SAM 2.1 Tiny【比較候補】

SAM 2.1は画像・動画のプロンプト型セグメンテーションに対応する。Tiny / Small / Base+ / Largeのチェックポイントがあり、画像APIではクリック・矩形プロンプ

トを利用できる。Apache 2.0で公開されている。

ただし、今回必要なのは静止画像編集だけであり、動画向け機能は不要である。MobileSAMで品質が不足した場合に、**SAM 2.1 Hiera Tiny**を比較する。

参考： SAM 2 GitHub

2.4 SAM 3 / SAM 3.1【不採用】

SAM 3はテキストプロンプトによる物体検出・セグメンテーションまで可能だが、公式実装はPython 3.12、PyTorch 2.7以上、CUDA対応GPUを前提とし、モデルも約848Mパラメータである。

今回の「クリックしてレイヤー分離」という要件に対して過剰であり、macOS中心・7週間という条件では導入リスクが高い。そのためVersion 1では利用しない。

参考： SAM 3 GitHub

2.5 Apple Siliconでの実行

PyTorchはmacOSのMetal Performance Shadersを利用する`mps`デバイスを公式に提供している。

Python側は以下の優先順位でデバイスを選択する。

1. `mps`
2. `cpu`

モデル固有の演算がMPSで正常動作しない場合には、自動的にCPUへフォールバックできる設計とする。

参考： PyTorch MPS backend

2.6 PoC合格条件

以下を実機で確認する。

- PNG/JPEGを1枚読み込める
- 画像上の1点を与えてマスクを生成できる
- 人物、動物、日用品の最低3種類を分離できる
- マスクから透明PNGを生成できる
- AI失敗時に例外でプロセス全体が終了しない
- 初回画像解析：目標5秒以内

- 同一画像への追加クリック：目標1秒以内

最終判定：MobileSAMを条件付き採用する。 最高スコアを自動確定せず最大3候補から選択可能とし、正・負クリックで補正できるUIを実装する。

---

3. JavaとAI処理の接続方式

3.1 Javaから直接ONNX Runtime【本線では不採用】

ONNX RuntimeにはJava APIが存在するが、公式Java CPUアーティファクトの対応プラットフォームにはWindows x64、Linux x64、macOS x64が記載されており、macOS ARM64は公式配布対象として明示されていない。

Apple Silicon向けネイティブライブラリを自前ビルドする方法はあるが、AI機能のためだけにJNI・ネイティブビルド問題を抱えるのは7週間の課題としてリスクが高い。

参考： ONNX Runtime Java

3.2 ローカルPythonサービス【採用】

JavaFX本体とAI処理を分離する。

```
JavaFX
  ↓ HTTP (127.0.0.1 only)
Local AI Service (Python)
  ↓
MobileSAM / PyTorch MPS
```

候補API：

- `GET /health`：AIサービス起動確認
- `POST /image`：編集対象画像を登録・特徴量を計算
- `POST /segment`：クリック座標・正負ラベルを送りマスクを返す
- `DELETE /image/{id}`：キャッシュ破棄

重要方針

- 外部ネットワークへ画像を送信しない
- サーバーは`127.0.0.1`のみでlistenする
- Java起動時に`ProcessBuilder`でPythonサービスを起動する方式を検討する

- Java側はAIサービスが起動していなくても通常編集を利用可能にする
- AI要求にはタイムアウトを設定する

最終判定：localhost HTTP方式を正式採用する。

---

4. PSD読み込み

4.1 Apache Commons Imaging【不採用】

Commons ImagingはPSDを読めるが、公式の対応表ではPSDレイヤー読み込みを十分に扱えず、基本的なPSD対応に留まる。レイヤー編集を目的とする本システムには不足する。

参考： Commons Imaging format support

4.2 TwelveMonkeys ImageIO【Javaのみの場合の代替候補】

TwelveMonkeysはJava ImageIO用PSDプラグインで、PSD/PSB読み込みとレイヤーデータへのアクセスを実装している。BSDライセンスで利用できる。

一方、レイヤー種別の取得などには未対応部分があり、テキストレイヤー・スマートオブジェクト等をPhotoshop同様に扱う用途には不十分である。

参考： TwelveMonkeys

4.3 psd-tools【正式採用】

既にAI用Pythonサービスを採用するため、PSDも同サービスに担当させる案が最も実装負荷が低い。

`psd-tools`はPSD/PSBのレイヤーツリーを読み込み、PixelLayer、TypeLayer、ShapeLayer、SmartObjectLayer、AdjustmentLayer等の種別を取得できる。各レイヤーを画像として出力することも可能である。ライセンスはMIT。

ただし、テキスト・シェイプ・スマートオブジェクト等の完全な編集やPhotoshop同等の合成はサポートされない。そのため、Version 1では以下に限定する。

PSD対応範囲

- レイヤー名取得
- レイヤー階層取得

- 表示状態取得
- レイヤー位置・サイズ取得
- ピクセルとして合成可能なレイヤーをPNG化
- Java側では原則RasterLayerとして取り込む
- 非対応レイヤーは警告を表示する
- 元PSDの完全再現・完全再保存は要求しない

参考： psd-tools

最終判定：psd-toolsを正式採用する。 実機PoCで公式PSD 5種類の階層・位置・表示状態・RGBA抽出を確認済み。Version 1では内部RasterLayerへの取り込みを対象とし、完全なPSD再保存は行わない。

---

5. JavaFX画像処理・表示

5.1 SwingFXUtils

`SwingFXUtils` は `BufferedImage` とJavaFX `Image` を相互変換できるが、変換時にはピクセルをコピーする。したがって、ドラッグやペン入力のたびに変換する設計は避ける。

参考： SwingFXUtils

5.2 基本方針

編集データの基準を `BufferedImage` とする。

理由：

- Javaの画像処理APIと相性が良い
- PNG/JPEGとの入出力が容易
- ピクセル操作、ぼかし、モザイク等の実装がしやすい

JavaFX側には表示用の `WritableImage` をキャッシュし、画像内容が変更されたタイミングだけ更新する。

5.3 性能不足時の最適化

JavaFXの `PixelBuffer` はアプリケーション側の `IntBuffer` / `ByteBuffer` を `WritableImage` と共有でき、更新時のコピーを減らせる。

初期実装では複雑化を避け、必要な場合のみPixelBuffer方式へ移行する。

参考： PixelBuffer

5.4 長時間処理

以下はJavaFX Application Threadで実行しない。

- 大画像フィルター
- PSD解析
- ファイル保存
- AI通信
- 発展機能の一括処理

`javafx.concurrent.Task` または `ExecutorService` を利用し、完了時だけUIへ反映する。

参考： JavaFX Task

一次判定：BufferedImage＋表示キャッシュ＋バックグラウンド処理を採用。

---

6. Undo／Redo

6.1 問題

4K RGBA画像は1枚あたり約32MiBとなるため、操作ごとに画像全体を保存すると履歴だけで数百MiB〜GiB単位まで増加する可能性がある。

6.2 採用方式

Commandパターン＋差分スナップショットのハイブリッドとする。

画像コピーを不要とする操作

状態値だけを保存する。

- レイヤー移動
- レイヤー順序変更
- 表示・非表示
- 透明度変更
- レイヤー名変更

- レイヤー追加／削除
- AIレイヤー生成（Undo時に生成レイヤーを削除）

画像データ保存が必要な操作

変更前の対象レイヤー、または変更領域のみを保存する。

- ペン
- 消しゴム
- フィルター
- ぼかし
- モザイク
- 色調整

6.3 履歴上限

`HistoryManager` にメモリ予算を設定する。

初期案：

- 最大履歴メモリ：256MiB
- 上限超過時は古い履歴から破棄
- 画面にUndo可能回数を保証しない

実機PoCでメモリ使用量を確認し、256〜512MiBの範囲で調整する。

一次判定：ハイブリッド方式を採用。

---

7. ライセンス確認

候補	ライセンス	判定
MobileSAM	Apache-2.0	利用可能
SAM 2	Apache-2.0	比較候補として利用可能
EfficientSAM	Apache-2.0	代替候補として利用可能
psd-tools	MIT	利用可能
TwelveMonkeys	BSD	利用可能

READMEまたはThird-party licensesファイルに使用ライブラリとライセンスを記載する。

---

8. 実機PoCで残っている検証

資料上の実現性は確認できたため、次は小さな試作で以下を計測する。

PoC-A AI

- MobileSAMをmacOSで起動
- MPS利用可否確認
- 1920×1080画像の初回解析時間
- クリック後のマスク生成時間
- 人物・動物・日用品で品質確認
- CPUフォールバック確認

不合格時：SAM 2.1 Tiny → EfficientSAMの順で比較する。

PoC-B PSD

最低3種類のPSDで確認する。

1. 通常のラスターレイヤーのみ
2. グループ付きPSD
3. テキスト／調整レイヤーを含むPSD

確認項目：

- レイヤー名
- 順序
- 表示状態
- 位置
- 透明度
- PNG化したレイヤーの見た目
- 非対応レイヤー検出

PoC-C JavaFX

対象画像：1920×1080、3840×2160。

計測する処理：

- 読み込み
- BufferedImage→JavaFX表示更新
- グレースケール
- モザイク
- ぼかし
- PNG保存

評価基準：

- UIがフリーズしない
- 2K画像の一般的な処理は概ね1秒以内を目標
- 4K画像で重い処理が遅い場合は、操作中は縮小プレビュー、確定時に原寸処理する方式へ変更する

PoC-D Undo／Redo

- 2K・4K画像で30回程度の編集を行う
- ヒープ使用量を確認する
- 256MiBの履歴予算で実用的なUndo回数を確保できるか確認する

---

9. 暫定技術スタック

```
Desktop App
  Java
  JavaFX
  BufferedImage / ImageIO
  Command-based History

Local Helper Service
  Python 3.10+
  FastAPI候補
  PyTorch
  MobileSAM
  psd-tools
```

```
Communication
  HTTP / localhost (127.0.0.1)
  JSON + PNG / multipart
```

最終確定条件

PoC-A〜Dが合格した時点で、上記構成を詳細設計の正式な技術スタックとして確定する。

---

10. 実機PoC結果（2026-10-06）

最終判定

PoC-A〜Dはすべて合格。暫定技術スタックを正式採用する。PoC-Aだけは候補マスク選択UIとMPS失敗時のCPU再試行を実装条件とする。

PoC-A：MobileSAM【条件付き採用】

- 実機：Apple M5 Pro（Apple Silicon）、macOS 26.5.1、PyTorch 2.14.0
- MPSコールド時：モデル読込 344ms、画像埋め込み 1,686ms、初回クリック 1,186ms。初回結果まで合計約3.22秒で目標5秒以内
- MPSウォーム時：画像埋め込み 353ms、初回クリック 60ms、追加クリック 33ms
- CPUフォールバック時：画像埋め込み 1,420ms、初回クリック 118ms、追加クリック 120ms
- 人物と物体をクリック指定し、アルファ付き透明PNGの生成に成功
- MPSとCPUでマスク面積・スコアは実用上同一

問題点：MobileSAMは1クリックで3候補を返す。バス画像では最高予測スコアが車体全体ではなく一部分を選んだ。したがって、最高スコアを自動確定せず、最大3候補のプレビュー選択、または正負の追加クリックで補正できるUIを採用する。モデルは常駐し、画像埋め込みをキャッシュし、起動時にダミー推論でウォームアップする。MPS推論に失敗した場合はCPUへ切り替えて1回だけ再試行する。

PoC-B：psd-tools【採用】

公式テストPSD 5種類で、レイヤー名、階層、位置、自己表示状態、親を含む実効表示状態、透明度、合成モード、レイヤー種別を取得できた。通常レイヤー、グループ、非表示グループ、Unicode名、TypeLayer、SmartObjectLayerを含め、全レイヤーのRGBA画像抽出と文書合成に成功し、抽出エラーは0件。

Version 1はメタデータと階層を保持し、表示・編集対象はRasterLayerへ変換する。未対応の調整・効果はフラット化して警告し、PSDへの完全再保存は対象外とする。

PoC-C：JavaFX / BufferedImage【採用】

中央値：2Kでは輝度処理3.6ms、3×3ぼかし23.9ms、BufferedImage→キャッシュ済みWritableImage更新2.8ms。4Kでは輝度8.1ms、ぼかし53.9ms、同変換6.3ms。4K ARGBの生データは約31.64MiB。

BufferedImage TYPE_INT_ARGB_PREを正本とし、WritableImageを再利用する。4K全体処理、PSD解析、保存、AI通信はTask / ExecutorServiceへ分離する。現時点ではPixelBuffer導入は不要。

PoC-D：Undo / Redo【採用】

4K画像で30操作を測定。全面スナップショットは約1,898MiBで不採用。512×512差分30回はペイロード60MiBに対して保持ヒープ約119.5MiB、256×256差分30回はペイロード15MiB・保持ヒープ約14.5MiB。256×256差分10回＋メタデータ20回はペイロード約5MiB・保持ヒープ約4.5MiB。Undo / Redoでスタック間を移動してもヒープは安定した。

Command＋最大256×256差分タイルを正式採用する。履歴上限は実データ量基準で256MiB。決定的な処理は変更前データ＋処理パラメータを保存し、Redo時に再計算して保存量を抑える。

総合結論

JavaFX＋BufferedImage＋Command履歴と、127.0.0.1限定Pythonサービス（MobileSAM＋psd-tools）の構成で実装へ進む。現時点で詳細設計を止める技術的な問題はない。